your Razor Pages will follow a consistent layout, making it easier for others to understand how each Razor Page in your app works.

NOTE This advice is just personal preference, so feel free to adapt it if there are aspects you don't agree with. The important thing is to understand why I make each suggestion, and to take that on board. Where appropriate, I deviate from these guidelines too!

Model binding in ASP.NET Core has a lot of equivalent approaches to take, so there is no "correct" way to do it. Listing 16.8 shows an example of how I would design a simple Razor Page. This Razor Page displays a form for a product with a given ID and allows you to edit the details using a `POST` request. It's a much longer sample than we've looked at so far, but I highlight the important points.

Listing 16.8 Designing an edit product Razor Page

```
public class EditProductModel : PageModel
{
    private readonly ProductService _productService;                ❶
    public EditProductModel(ProductService productService)          ❶
    {                                                               ❶
        _productService = productService;                           ❶
    }                                                               ❶

    [BindProperty]                                                  ❷
    public InputModel Input { get; set; }                           ❷

    public IActionResult OnGet(int id)                              ❸
    {
        var product = _productService.GetProduct(id);               ❹

        Input = new InputModel                                      ❺
        {                                                           ❺
            Name = product.ProductName,                             ❺
            Price = product.SellPrice,                              ❺
        };                                                          ❺
        return Page();                                              ❺
    }
```